Mid-semester presentation · 20 minutes + 10 minutes of questions

Presentation script

What each of us says on each slide. Read it aloud a few times so it sounds natural, then speak from memory; don’t read from the paper on the day.

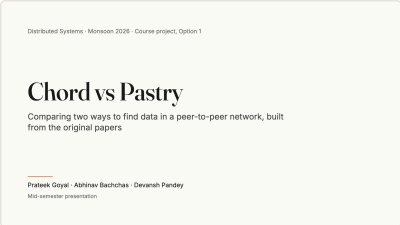
Speaker	Slides	Time	Must also know (study guide)
Prateek Goyal	1–5 (problem, motivation, Chord) and 9 (related work)	about 8 min 20 s	Questions 1–52 and 81–86
Devansh Pandey	6–8 (Pastry, side by side)	about 4 min 25 s	Questions 53–80
Abhinav Bachchas	10–15 (our work, results, summary)	about 6 min 35 s	Questions 87–124

Running clock

Slide	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
starts at	0:00	0:20	1:40	3:00	4:50	6:30	8:20	10:00	10:55	12:45	14:25	15:35	17:00	18:10	19:05

Total about 19 min 20 s, which leaves a little room. If you are behind at slide 9 (should start by 11:00), shorten slide 9 to the first and last paper.

Tips. Face the audience, not the screen. Point at the figure when the script says “point at”. Say numbers slowly. The orange handover line tells the next speaker it is their turn. If you are asked something during your part, answer in two or three sentences and continue.

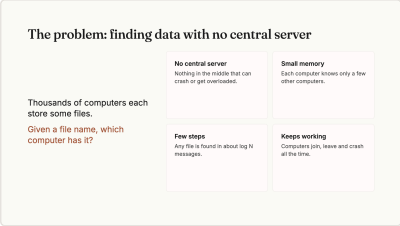


Prateek · 20 seconds

Slide 1 · Chord vs Pastry

Stand next to the screen. Introduce everyone by name.

Good morning. We are Prateek, Devansh and Abhinav. Our project is Chord versus Pastry: two famous ways to find data in a peer-to-peer network. We implemented both from their original papers and compared them in a simulator we wrote ourselves. I'll start with the problem.



Prateek · 1 min 20 s

Slide 2 · The problem: finding data with no central server

Point at the orange question, then at the four boxes one by one.

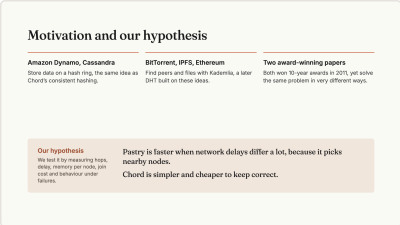
Imagine thousands of computers, and each one stores some files. There is no central server. Someone gives you a file name. The question is: which computer has that file?

The simple ideas don't work. A central directory, like Napster, is a single point of failure. Asking everyone, like Gnutella, sends far too many messages. And if every computer kept a list of everyone else, every join or crash would have to update thousands of lists.

So a good solution needs four things. No central server. Small memory: each computer knows only a few others. Few steps: any file is found in about log N messages. And it must keep working while computers join, leave and crash.

A system that does this is called a distributed hash table, or DHT. Every computer and every file name is hashed into the same range of numbers, and a fixed rule says which computer owns which key. Chord and Pastry are two different rules for this.

> If asked: basics such as DHT, consistent hashing, why not hash mod N, O(log N) are questions 1–20 in the guide.



Prateek · 1 min 20 s

Slide 3 · Motivation and our hypothesis

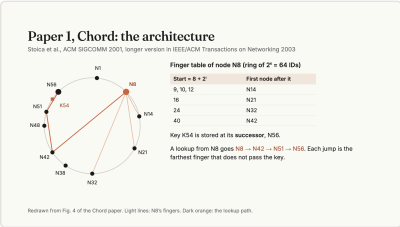
Point at the three systems, then read the hypothesis slowly.

Why does this matter today? These papers are from 2001, but the ideas run real systems. Amazon's Dynamo and Cassandra store data on a hash ring, which is Chord's idea. BitTorrent, IPFS and Ethereum find peers with Kademlia, a later DHT built on the same ideas. And both papers won ten-year awards in 2011.

We picked these two because they solve the same problem in very different ways. Chord uses a ring with shortcuts. Pastry matches digits and prefers nearby computers. A fair side-by-side test shows which design choices really matter.

Our hypothesis has two parts. First, Pastry is faster when network delays differ a lot, because it picks nearby nodes. Second, Chord is simpler and cheaper to keep correct. To test it we measure hops, delay, memory per node, join cost, and what happens when nodes fail.

> If asked "why this project / what is the benefit / why old papers": questions 21–32.



Prateek · 1 min 50 s

Slide 4 · Paper 1, Chord: the architecture

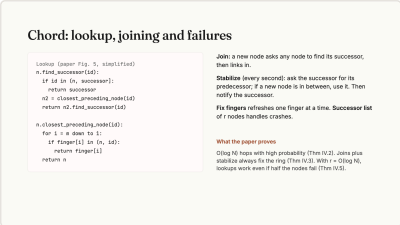
Point at K54 and N56, then at N8's light finger lines, then follow the orange path N8 → N42 → N51 → N56 with your hand.

This is Chord's architecture, redrawn from Figure 4 of the Chord paper. All IDs sit on a ring. Here the ring has 64 positions and ten nodes. A key is stored at its successor, the first node at or after it going clockwise. So key 54 is stored on node 56.

If each node only knew its next neighbour, a search would walk around the whole ring. So each node keeps a finger table. Finger i points to the first node at least 2 to the power i positions ahead. For node 8 that gives nodes 14, 21, 32 and 42: nodes a half, a quarter, an eighth of the way round.

Now a lookup for key 54 from node 8. Node 8 jumps to its farthest finger that does not pass 54, which is node 42. Node 42 does the same and jumps to 51. Node 51 sees that 54 lies between itself and its successor, so the answer is node 56. Three hops. Each jump at least halves the remaining distance, so a lookup takes about log N hops.

> If asked "why ½·$\log_2 N$ on average" or "why closest preceding": questions 34–37.



Prateek · 1 min 40 s

Slide 5 · Chord: lookup, joining and failures

Point at the first “if” in the code, then at Stabilize on the right.

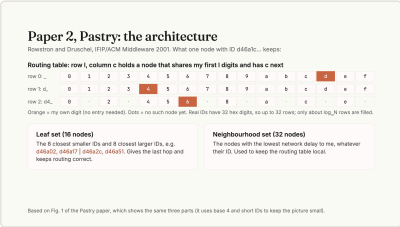
On the left is the lookup code from the paper, slightly simplified. If the key is between me and my successor, my successor owns it. Otherwise I pass the request to my closest preceding finger: the farthest finger that is still before the key.

On the right is how Chord stays correct while nodes come and go. A new node only needs to find its successor. Then every node runs stabilize about once a second: it asks its successor, “who is your predecessor?” If a new node has joined in between, it switches to that node and notifies it. Fix fingers updates one finger at a time. And each node keeps a list of r successors, so if one crashes, the next one takes over.

The paper proves three main results: lookups take O(log N) hops; stabilize always repairs the ring after joins; and with a successor list of order log N, lookups still work even if half the nodes fail.

Now Devansh will explain Pastry.

> If asked about stabilize, notify, concurrent joins, successor list length, crashes, leaving: questions 38–52.



Devansh · 1 min 50 s

Slide 6 · Paper 2, Pastry: the architecture

Point at row 0, row 1, row 2 and the orange cells; then the two boxes at the bottom.

Thank you. Pastry was published by Rowstron and Druschel at Middleware 2001. This slide shows what one Pastry node keeps; it is the same three parts as Figure 1 in the paper.

IDs are 128-bit numbers, read as 32 hex digits. A key belongs to the node whose ID is numerically closest to it.

The main part is the routing table. Row 0 has one node for each possible first digit. Row 1 has nodes that share my first digit, d, one for each second digit. Row 2 shares d4, and so on. The orange cell is my own digit, so it needs no entry. Lower rows are emptier, because fewer nodes share a long prefix, so only about log base 16 of N rows are filled.

The leaf set holds the 16 nodes with the closest IDs, 8 smaller and 8 larger. It is used for the last hop, and it is what keeps routing correct. The neighbourhood set holds the 32 nodes with the lowest delay to me, whatever their ID. It helps keep the routing table full of nearby nodes.

> If asked about b, the empty column, leaf set or neighbourhood set: questions 54–58.



Devansh · 1 min 40 s

Slide 7 · Pastry: routing, joining and locality

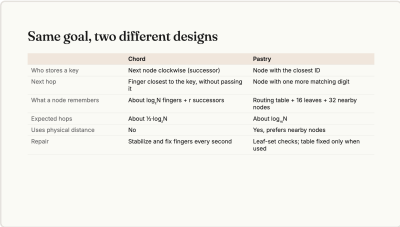
Point at each of the three branches of the code, then follow the example route with your hand.

This is Pastry's routing rule, Table 1 in the paper, simplified. If the key is inside my leaf set range, I send it to the leaf with the closest ID. Otherwise I count how many starting digits the key shares with my ID; call it l. I look in row l for the key's next digit, and that node shares one more digit with the key. If that entry is empty, the rare case, I send it to any node I know that is at least as good in prefix and numerically closer.

On the right is an example route for key d46a1c. Each hop matches one more digit: d, then d4, then d46. The last hop uses the leaf set. Because each hop fixes one hex digit, a lookup takes about log base 16 of N hops.

To join, a new node routes a message to its own ID, starting from a nearby node, and collects table rows from each node on the way. And locality: for every table entry, Pastry keeps the node with the lowest delay. That is why most of its hops are short.

> If asked about the rare case, why it always finishes, failures, join details, the second join step: questions 59–72.



Devansh · 55 seconds

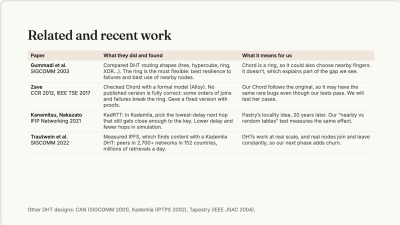
Slide 8 · Same goal, two different designs

Go through the table row by row, briefly.

Here are the two designs side by side. Chord stores a key at its successor; Pastry at the closest ID. Chord's next hop is the farthest finger before the key; Pastry's is a node with one more matching digit. Chord remembers about log N fingers; Pastry remembers more: a routing table, 16 leaves and 32 nearby nodes. In return Pastry needs fewer hops, about log base 16 of N instead of half of log base 2 of N, and it uses physical distance, which Chord does not.

Now Prateek will cover the related work.

> If asked "which is better" or "why fewer hops": questions 73–80.



Prateek · 1 min 50 s

Slide 9 · Related and recent work

One row at a time: say the name and year, what they found, then what it means for us.

We reviewed four papers around our topic.

Gummadi and others, SIGCOMM 2003, compared the routing shapes that DHTs use. They found the ring is the most flexible: it gives the best resilience to failures and the best use of nearby nodes. So Chord, which is a ring, could also choose nearby fingers. The original Chord doesn't, and that explains part of the gap we measure.

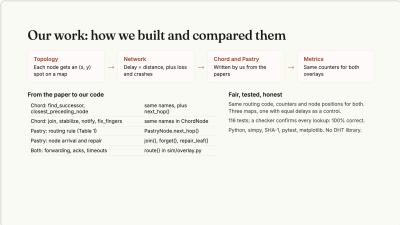
Pamela Zave, in 2012 and 2017, checked Chord with a formal model and showed that the original protocol is not fully correct: some orders of joins and failures break the ring. She published a fixed version with proofs. Our Chord follows the original, so we plan to test her cases.

KadRTT, IFIP Networking 2021, changes Kademlia to pick the lowest-delay next hop that still gets close to the key. It lowers both delay and hops. That is Pastry's locality idea, twenty years later.

And the IPFS paper, SIGCOMM 2022, measured a real DHT with peers in 152 countries. It shows that nodes come and go all the time, which is why we add churn next.

Now Abhinav will explain what we built and what we found.

> If asked about any of these papers or other DHTs: questions 81–86.



Abhinav · 1 min 40 s

Slide 10 · Our work: how we built and compared them

Follow the four boxes left to right, then point at the table.

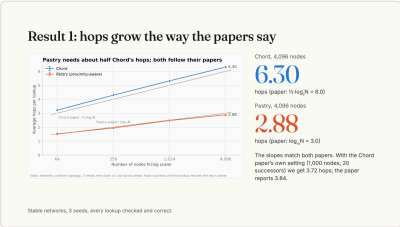
Thanks. We built everything as a simulation on one laptop, in Python with simpy, a discrete-event simulator. Each node gets a position on a map, and a message's delay is the distance between the two nodes. The network can also lose messages and crash nodes.

We wrote Chord and Pastry ourselves from the papers. The table shows the paper's functions next to ours: Chord's find_successor, stabilize and notify keep the same names, and Pastry's routing rule is our next_hop function.

To make the comparison fair, both use the same forwarding and timeout code, the same counters, and the same node positions. We test on three maps; on one of them every link has the same delay, so picking nearby nodes can't help. That map is our control.

We have 116 tests, and a checker that knows the correct owner of every key confirms every lookup. All of them were correct.

> If asked about simpy, yield, timeouts, the oracle, fairness, what we changed, the bug we fixed: questions 87–100.



Abhinav · 1 min 10 s

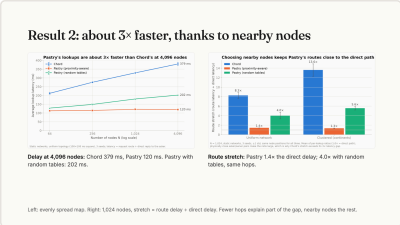
Slide 11 · Result 1: hops grow the way the papers say

Point at the x-axis, the grey lines, then the two big numbers.

First result: hops. The x-axis is the number of nodes, from 64 to 4,096. The grey lines are what the papers predict. At 4,096 nodes Chord needs 6.30 hops and Pastry 2.88, so Pastry needs about half.

Both follow their papers. Chord adds half a hop each time the network doubles; Pastry adds one hop each time it grows 16 times. We also checked a number printed in the Chord paper: with 1,000 nodes and 20 successors the paper reports 3.84 hops, and we get 3.72 with the same setting.

> If asked "why 6.30 and not 6.0": question 102.



Abhinav · 1 min 25 s

Slide 12 · Result 2: about 3× faster, thanks to nearby nodes

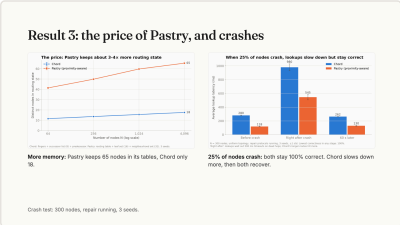
Left graph first (three lines), then the orange and green bars on the right.

But fewer hops does not automatically mean faster, so here is the real delay. On the left, at 4,096 nodes Chord takes 379 milliseconds and Pastry 120: about three times faster.

To find out why, we also ran Pastry with random tables: the same hops, but no preference for nearby nodes. It takes 202 milliseconds. So part of the gap is fewer hops, and the rest comes from choosing nearby nodes.

On the right is stretch: how much longer the route is than sending directly. Pastry is only 1.4 times the direct delay; with random tables it is 4. On the clustered map, where nodes form continents, the difference is even bigger. Chord never looks at distance, so its stretch is much higher.

> If asked “why is Chord’s stretch 8× when delay is only 3×”: question 103.



Abhinav · 1 min 10 s

Slide 13 · Result 3: the price of Pastry, and crashes

Left graph, then the tall middle bars in the right graph.

Pastry’s speed has a price. On the left: each Pastry node remembers about 65 other nodes, Chord only about 18. More state means more to keep up to date.

On the right we crashed 25 percent of the nodes at once. Right after the crash, lookups get slower because they wait for timeouts on dead nodes, and Chord suffers more because its routes are longer. But every lookup still found the right owner, and after 60 seconds both were back to normal. Chord also cleaned up its tables faster than Pastry.

> If asked about crashes, join cost or message loss: questions 104, 107, 108.



Abhinav · 55 seconds

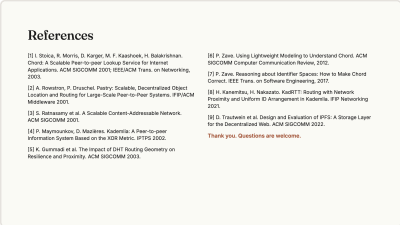
Slide 14 · Summary, limits and next steps

Read the three numbers slowly. Say the limits plainly; it shows you understand your own work.

To sum up: Pastry needs fewer hops, 2.88 against 6.30, and is about three times faster, 120 against 379 milliseconds. Chord remembers far fewer nodes, 18 against 65, and repairs faster. So our hypothesis holds so far.

We are honest about the limits: there is no churn yet, our Pastry reads delays for free, we used only three seeds, and we have not yet tested Zave’s cases. Next we add the key-value store and churn, run the full experiments, and write the report.

> If asked “what is bad about your project”, “is the simulation realistic”, “is it fair to Chord”: questions 106 and 109–118.



Abhinav · 15 seconds

Slide 15 · References

All three stand together for questions.

These are our references. Thank you, we are happy to take your questions.

During questions. The person whose slides the question is about answers first; others add only if needed. Keep answers to two or three sentences, give a number when you can, and say “we haven’t tested that yet” when that is the truth. All answers are in Part 2 of the study guide.